

DNA Self-Assembly For Constructing 3D Boxes

(Extended Abstract)

Ming-Yang Kao^{1,*} and Vijay Ramachandran^{2,**}

¹ Department of Computer Science, Northwestern University
Evanston, IL 60201, USA
`kao@cs.northwestern.edu`

² Department of Computer Science, Yale University
New Haven, CT 06520-8285, USA
`vijayr@cs.yale.edu`

Abstract. We propose a mathematical model of DNA self-assembly using 2D tiles to form 3D nanostructures. This is the first work to combine studies in self-assembly and nanotechnology in 3D, just as Rothemund and Winfree did in the 2D case. Our model is a more precise superset of their Tile Assembly Model that facilitates building scalable 3D molecules. Under our model, we present algorithms to build a hollow cube, which is intuitively one of the simplest 3D structures to construct. We also introduce five basic measures of complexity to analyze these algorithms. Our model and algorithmic techniques are applicable to more complex 2D and 3D nanostructures.

1 Introduction

DNA *nanotechnology* and DNA *self-assembly* are two related technologies with enormous potentials.

The goal of DNA nanotechnology is to construct small objects with high precision. Seeman's visionary work [8] in 1982 pioneered the molecular units used in self-assembly of such objects. More than a decade later, double-crossover (DX) molecules were proposed by Fu and Seeman [3] and triple-crossover (TX) molecules by LaBean *et al.* [5] as DNA self-assembly building blocks. Laboratory efforts have been successful in generating interesting three-dimensional (3D) molecular structures, including the small cube of Chen and Seeman [1]. However, these are immutable and limited in size, mainly because their fabrication is not based on a mathematical model that can be extended as necessary.

In parallel to DNA nanotechnology, studies on self-assembly of DNA tiles have focused on using local deterministic binding rules to perform computations. These rules are based on interactions between exposed DNA sequences

* Supported in part by NSF Grants CCR-9531028 and EIA-0112934. Part of this work was performed while this author was visiting the Department of Computer Science, Yale University, New Haven, CT 06520-8285, USA, `kao-ming-yang@cs.yale.edu`.

** Supported by a 2001 National Defense Science and Engineering Graduate Fellowship.

on individual tiles; tiles assemble into a particular 1D or 2D structure when in solution, encoding a computation. Winfree [10] formulated a model for 2D computations using DX molecules. Winfree *et al.* [11] used 1D tiles for 1D computations and 2D constructions with DX molecules. LaBean *et al.* [4] were the first to compute with TX molecules.

Combining these two technologies, several researchers have demonstrated the power of DNA self-assembly in nanostructure fabrication. Winfree *et al.* [13] investigated how to use self-assembly of DX molecules to build 2D lattice DNA crystals. Rothemund and Winfree [7] further proposed a mathematical model and a complexity measure for building such 2D structures.

A natural extension of the seminal 2D results of Winfree *et al.* [13] and Rothemund and Winfree [7] would be the creation of 3D nanostructures using tiling. To initiate such an extension, this paper (1) proposes a general mathematical model for constructing 3D structures from 2D tiles; (2) identifies a set of biological and algorithmic issues basic to the implementation of this model; and (3) provides basic computational concepts and techniques to address these issues. Under the model, the paper focuses on the problem of constructing a hollow cube, which is intuitively one of the simplest 3D structures to construct. We present algorithms for the problem and analyze them in terms of five basic measures of complexity.

There are three natural approaches to building a hollow cube. The first approach uses 1D tiles to form 2D DX-type tiles as in [11], and then uses these tiles to construct a cube. Our paper does not fully investigate this possibility because of the inconvenient shape of these molecules (see Sect. 2.1), but our algorithms can be modified to accommodate these DX-type tiles. The second approach builds a cube from genuine 2D tiles, which is the focus of this paper. The third approach is perhaps the most natural: build a cube from genuine 3D tiles. It is not yet clear how such 3D tiles could be created; conceivably, the cube of Chen and Seeman [1] may lead to tiles of this form. This paper does not fully investigate this possibility, either, because this approach is algorithmically straightforward and similar to the 2D case.

The basic idea of our algorithms is to use 2D tiles to form a shape on the plane that can fold into a box, as illustrated in Fig. 1(a)–(b). We can easily synthesize a set of tiles to create the initial 2D shape. To overcome a negligible probability of success due to biochemical factors, we must put many copies of these tiles into solution at once; but we must then worry about multiple copies of the shape interfering with each other, preventing folding, as in Figure 1(c).

To avoid this problem, we introduce randomization, so that different copies of the shape have unique sticky ends. The growth of tiles into a complete structure must still be deterministic (as it is based on Watson-Crick hybridization), but we randomize the computation input — the *seed tiles* from which the rest of the shape assembles. The edges then still relate to each other, but depend on the random input that is different for each shape in solution. If each input can form with only low probability, interference with another copy of the shape will be kept to a minimum.

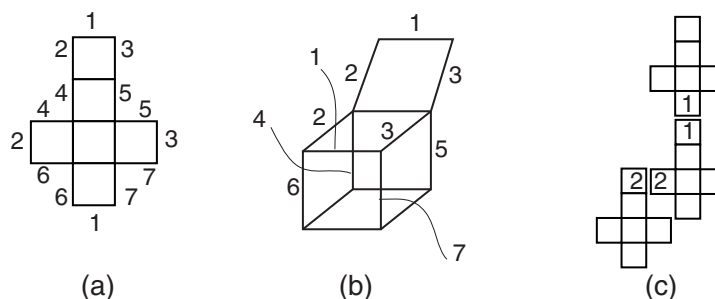


Fig. 1. (a) 2D planar shape that will fold into a box. Each section is formed from many smaller 2D DNA tiles. Edges with the same number have complementary sticky ends exposed so they can hybridize. (b) Folding of the shape in (a) into a box. Here, edges 4, 5, 6, and 7 have all hybridized. Hybridization of edges 2 and 3, whose two complements are now in close proximity, will cause edge 1 to hybridize and form the complete box. (c) Multiple copies of the 2D shape in solution. Copies of the shape can interfere and attach infinitely without control as long as edges have matching sticky ends

This raises another important issue — that of using self-assembly to communicate information from one part of the shape to another. Since the edges must relate to each other and the random input, designing local rules becomes nontrivial. In this paper, we explore and formalize patterns used in completing this task. In addition, we formalize biological steps that allow a specific subset of tiles to be added in an isolated period of time, thus allowing better control of growth. We couple this with the use of temperature to improve the probability of a successful construction.

The remainder of this paper is organized as follows. Sect. 2 describes the model of computation, including notation for DNA tiles and definitions of complexity measures. Sect. 3 describes the algorithms in detail, and Sect. 4 discusses future research possibilities. Some details have been omitted from this extended abstract; the full version is available electronically at

http://www.cs.yale.edu/~vijayr/papers/dna_3d.ps.

2 Model of Computation

In this section we formally introduce our model of self-assembly, the *Generalized Tile Assembly Model*, on both the mathematical and biological level. It is an extension of the model presented by Rothmund and Winfree in [7].

2.1 Molecular Units of Self-Assembly

We begin with the biological foundation for our model. We intend to build 3D structures using the folding technique shown in Fig. 1 and allow construction of all 2D structures possible with the Tile Assembly Model.

Our model relies on using the molecular building block of a *DNA tile*. Tiles can naturally hybridize to form stable shapes of varying sizes, and the individual tiles can easily be customized and replicated (via synthesis and PCR before the procedure) for a specific algorithm.

DNA tiles are small nucleotides with exposed action sites (also known as sticky ends of a DNA strand) consisting of a single-stranded sequence of base pairs. When this sequence matches a complementary sequence on an action site of another tile, the Watson-Crick hybridization property of DNA causes these two molecules to bind together, forming a larger structure. A tile can be synthesized in the laboratory to have specific sticky ends. Different combinations of sticky ends on a tile essentially yield uniquely-shaped puzzle pieces. The tiles will automatically hybridize when left in solution.

Most work in self-assembly uses DX and TX molecules for tiles, but the shape of these molecules causes a problem for 3D construction. Since the sticky ends are on diagonally opposite ends (see [3] and [5]), these tiles form structures with ragged edges when they hybridize, as in Fig. 2(a). Our algorithms can easily be modified to use these tiles by adjusting for proper alignment before folding into a box.

However, we propose a simpler alternative, which is using the branched molecules of Seeman [8] or a variant derived from the structure of tRNA. These molecules, sketched in Fig. 2(b) and (c), are truly 2D with sticky ends on four sides. The structure is stable while the sticky ends are free-floating in solution — so the molecules have flexibility to align properly during folding.

Such molecules offer a natural motivation for modeling them using Wang's theory of tiling [9], which allows us to abstract construction using these molecules to a symbolic level.

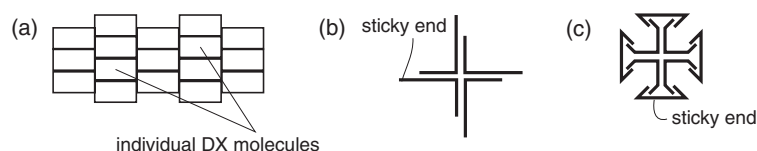


Fig. 2. (a) 2D structure formed from DX molecules. The left and right sides cannot hybridize because they are aligned improperly; the same is true for the top and bottom sides. (b) Branched-molecule DNA tile from [8]. (c) Synthetic DNA tile derived from the structure of tRNA

2.2 Symbolic Representation of Tiles

Definition 1. A DNA sequence of length n is an ordered sequence of base pairs $5' - b_1 b_2 \cdots b_n - 3'$ where the sequence has a 5-prime and 3-prime end, and $b_i \in \mathcal{B} = \{A, T, C, G\}$, the set of base pairs. We will assume that if the directions are not explicitly written, the sequence is written in the $5' \rightarrow 3'$ direction.

1. The Watson-Crick complement of sequence $s = 5' - b_1 b_2 \cdots b_n - 3'$, denoted $\bar{s}$, is the sequence¹ $3' - \bar{b}_1 \bar{b}_2 \cdots \bar{b}_n - 5'$, where $\bar{A} = T$, $\bar{C} = G$. Define $\overline{\bar{s}} = s$.
2. The concatenation of two sequences $s = s_1 \cdots s_n$ and $t = t_1 \cdots t_m$, denoted $s \cdot t$, or simply st , is the sequence $s_1 \cdots s_n t_1 \cdots t_m$.
3. The subsequence from i to j of sequence $s = 5' - b_1 b_2 \cdots b_n - 3'$, denoted $s[i : j]$, is the sequence $5' - b_i b_{i+1} \cdots b_{j-1} b_j - 3'$, where $1 \leq i < j \leq n$.

Given the above definitions, two DNA strands can hybridize if they have complementary sequences. Formally, $s = s_1 \cdots s_n$ and $t = t_1 \cdots t_m$ can hybridize if there exist integers $h_{s1}, h_{s2}, h_{t1}, h_{t2}$ such that $s[h_{s1} : h_{s2}] = \bar{t}[h_{t1} : h_{t2}]$. We assume there are no misbindings, that is, the above condition must be met exactly with no errors in base-pair binding.

Remark 1. Note that $\overline{st}$ (or $\overline{(s \cdot t)}$) $\neq \bar{s}\bar{t}$ (or $\bar{s} \cdot \bar{t}$); rather, $\overline{st} = \bar{t} \cdot \bar{s}$.

Definition 2. The threshold temperature for a DNA sequence is a temperature t in some fixed set $\mathcal{T}$ such that the sequence is unable to remain stably hybridized to its complement when the solution is at a temperature higher than $t' \in (t - \epsilon, t + \epsilon)$ for $\epsilon > 0$.² (Heating a solution generally denatures strands, so this definition has strong biological foundation. The consequences and methodology of using temperature in designing DNA sequences for tiles is discussed in [2].) If s has a lower threshold temperature than t , we say s binds weaker than t .

As with most work in DNA computing, our model uses DNA sequences to encode information³—in our case, an identifier specifying what kinds of matches are allowed between tiles on a given side. Since there are no misbindings, these identifiers map uniquely to DNA sequences present on the sides of tiles that can bind to each other. Formally, we have the following.

Definition 3. Let $\mathcal{S}$ be the set of symbols used to represent the patterns on the sides of our tiles. We assume $\mathcal{S}$ is closed under complementation, that is, if $s \in \mathcal{S}$ then there exists some $s' \in \mathcal{S}$ such that $s' = \bar{s}$ where $\bar{s}$ is the complement of s (the purpose of this will be clear below). Let $\mathcal{W} \subset \bigcup_i \mathcal{B}^i$ be the set of DNA

¹ The assumption that sequences written without directions are given $5' \rightarrow 3'$ means that the complement of $b_1 b_2 \cdots b_n$ is $\bar{b}_n \cdots \bar{b}_2 \bar{b}_1$, which is not standard convention but is technically correct.

² A fixed set of threshold temperatures simplifies the model and corresponds to the temperature parameter in [7]. To compensate we allow the actual threshold temperature to deviate slightly from the fixed point.

³ Condon, Corn, and Marathe [2] have done work on designing good DNA sequences for problems like this one.

sequences called DNA words such that the words do not interfere with each other or themselves (i.e., bind inappropriately). We then define the injective map $\text{enc} : \mathcal{S} \rightarrow \mathcal{W}$ that is the encoding of a symbol into a DNA word. This map obeys complementation: $\text{enc}(\bar{s}) = \text{enc}(s)$.

Definition 4. A DNA tile is a 4-tuple of symbols $T = (s_N, s_E, s_S, s_W)$ such that $s_i \in \mathcal{S}$ and $\text{enc}(s_i)$ is the exposed DNA sequence at the north, east, south, or west action site of the tile, for $i = N, E, S, W$. Given two tiles T_1 and T_2 , they will bind if two sides have complementary symbols. Properties of hybridization, including threshold temperature, carry over to the hybridization of tiles. We make a stronger no-misbinding assumption for tiles, requiring that the sticky ends on the tiles match exactly and fully.

At this stage, our model exactly matches that of Rothemund and Winfree in [7], except that our tiles can “rotate”; that is, $(s_N, s_E, s_S, s_W) = (s_E, s_S, s_W, s_N)$. This corresponds more closely to tile structure. The model, at this point, could require many symbols to express different tile types, and possibly an exponential number of DNA words. Ideally, we would like to arbitrarily extend the symbolic or informational content of each side of a tile. Therefore we make the following generalization.

Definition 5. Let Σ be a set of symbols closed under complementation, and let Ω be a set of corresponding DNA words. A k -level generalization of the model defines a map $g : \Sigma^k \rightarrow \mathcal{S}$ and a corresponding encoding $\text{genc} : \Sigma^k \rightarrow \mathcal{W}$, where $\text{genc}(\sigma) = \text{enc}(g(\sigma))$ for $\sigma \in \Sigma^k$ such that an abstract tile definition, which is a 4-tuple of k -tuples of symbols in Σ , is equivalent to a DNA tile.

We define complementation for a k -tuple in Σ^k as follows: let $\bar{\sigma} = (\bar{\sigma}_1, \dots, \bar{\sigma}_k)$ be $(\bar{\sigma}_1, \dots, \bar{\sigma}_k)$ so that $\text{genc}(\bar{\sigma}) = \text{enc}(g(\bar{\sigma})) = \text{enc}(g(\sigma)) = \text{enc}(g(\sigma))$. This makes the hybridization condition equivalent to having complementary symbols in k -tuples for sides that will bind.

The definition is purposefully broad in order to allow different algorithms to define the encoding based on the number of words and tiles needed. A 1-level generalization with $\Sigma = \mathcal{S}$ and $\Omega = \mathcal{W}$ where $g(s) = s$ and $\text{genc} = \text{enc}$ is the original Rothemund-Winfree Tile Assembly Model. In this paper, we use the following model.

Definition 6. The concatenation generalization is a k -level generalization where $\mathcal{W} \subset \Omega^k$ and g maps every combination of symbols in Σ^k to a unique symbol in $\mathcal{S}$. Partition $\mathcal{S}$ into $\mathcal{S}'$ and $\bar{\mathcal{S}}'$ such that each set contains the complement symbols of the other, and $\mathcal{S}' \cap \bar{\mathcal{S}}' = \emptyset$.⁴ For $\sigma \in \mathcal{S}'$, define $\text{genc}(g^{-1}(\sigma)) = \text{enc}(\sigma) = \omega_1 \omega_2 \dots \omega_k$, where $\text{enc}(\sigma_i) = \omega_i \in \Omega$ and $g^{-1}(\sigma) = (\sigma_1, \sigma_2, \dots, \sigma_k)$. Then for $\bar{\sigma} \in \bar{\mathcal{S}}'$, let $\text{enc}(\bar{\sigma}) = \bar{\omega}_k \cdot \bar{\omega}_{k-1} \dots \bar{\omega}_1$, so that $\text{genc}(g^{-1}(\bar{\sigma})) = \text{genc}(g^{-1}(\sigma))$.

⁴ The map enc as defined will be one-to-one, and so is g , and so this can be done since the DNA sequence corresponding to any symbol has a unique complement, and therefore a unique complement symbol.

In other words, the concatenation generalization model is a straightforward extension of the tile model where each side of a tile corresponds to a k -tuple of symbols, where the DNA sequence at the corresponding action site is simply the concatenation of the encodings of the individual symbols.⁵ Using this simple model, we can reduce the number of DNA words needed to $|\Sigma|$ from $|\Sigma|^k$, and create simpler descriptions of our tiles.

2.3 Algorithmic Procedures

With the above models for tiles, we now discuss procedures for growing larger structures.

We follow Rothmund and Winfree [7] and Markov [6] and use the common self-assembly assumption that a structure begins with a seed tile and grows, at each timestep, by hybridization with another free-floating tile.⁶ The new tile hybridizes at a given position following one of two types of rules:

Deterministic Given the surrounding tiles at that position, only one tile type, with specific sticky ends on the non-binding sides, can fit.

Randomized Multiple tile types (with different sticky ends on the non-binding sides) could fit the position given the tiles present; a new action site is created with probability proportional to the concentration of its tile type in solution.

Therefore, to grow a structure, an algorithm repeats *steps* until the structure is complete: add tiles to solution; wait for them to adhere to the growing structure; optionally removes excess tiles from solution by “washing them away.” Cycling temperature during these steps to prevent or induce binding (based on threshold temperatures) can be done while waiting for hybridization, and is called *temperature-sensitive binding*.

2.4 Complexity

We consider five basic methods of analyzing algorithms using our model.

Time complexity Each algorithm is a sequence of self-assembly steps described above, thus the natural measure of *time complexity* in our model is the number of steps required (which describes laboratory time).

Space complexity The number of distinct physical tile types (not the actual number of molecules produced) is *space complexity*. Introduced by [7], this describes the amount of unique DNA synthesis necessary.

⁵ Potentially, the concatenation model could cause interference among tiles. If we maintain no misbindings, however, our model removes this from analysis. In addition, it is theoretically possible to design DNA words so interference does not occur, depending on the algorithm.

⁶ In reality, multiple tiles can hybridize at once and structures consisting of more than one tile can hybridize to each other, but we lose no generality with the Markov assumption.

Alphabet size The number of DNA words, or $|\Omega|$ or $|\mathcal{W}|$, has a rough laboratory limit [2], and so the size of the symbol set used ($|\Sigma|$ or $|\mathcal{S}|$), which corresponds directly to the number of words, has practical significance.

Generalization level The generalization level is the amount of information on a side of a tile. This is related to the length of the sticky ends (and thus has biological consequences) and the number of actual DNA words (via $|\mathcal{S}|$).

Probability of misformation *Misformed structures* contain tiles that are not bound properly on all sides. Assuming the Markov model, consider adding tile T to a partial structure S . If complete hybridization requires binding on two sides, but T manages to hybridize only on one side (while the other action site does not match), $S + T$ has a *misformation*. We quantify this probability with the following.

Definition 7. Let the success probability at step t be the probability that a free-floating tile in solution binds at all possible sides to a partial structure at a given spot. (Step t is the addition of a tile to that spot on structure S_t , resulting in S_{t+1} .) This is

$$\Pr(S_{t+1} \text{ is correct} | S_t \text{ is correct}) = \frac{N_{\text{correct}}}{N_{\text{all}}},$$

where N_{correct} is the number of tile types that can correctly bind, while N_{all} is the number of tile types in solution that could bind, possibly even incompletely. Call this q_t . Then the misformation probability at step t is $p_t = 1 - q_t$. If the algorithm has k additions, then the misformation probability for the algorithm is $1 - q_0 q_1 \cdots q_{k-1}$. Then an algorithm is *misformation-proof* if its misformation probability at every step is zero, yielding a zero total probability of misformation.

3 Hollow Cube Algorithms

In this section, we examine algorithms designed to use our model to build a 3D hollow cube using the folding technique shown in Fig. 1. Let the length of a side of the cube, n , be the input to the algorithms. We present the most interesting algorithm in detail and defer the discussion of others considered to the full version of the paper.

3.1 Overview

Figure 3(a) illustrates the planar shape our algorithms construct. We will reference the labels and shading of regions in the figure during our discussion.

As stated earlier in Sect. 1, we must make each shape unique so different partial structures in solution do not bind to and interfere with each other. Once we have a unique seed structure, we can then use self-assembly with basic rules to make the edges of the shape correspond so folding will occur.

There are three basic self-assembly *patterns* used to construct different parts of the shape.

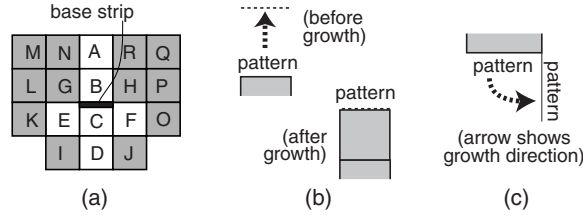


Fig. 3. (a) Regions of the 2D planar shape. (b) A straight-copy pattern. (c) A turn-copy pattern

Random assembly Implements a random rule (see Sect. 2.3). Formally, add all tiles in a set of distinct tiles R in equal concentrations so each could potentially hybridize completely at a given position. Thus the information at that position is completely random. The tiles differ by a component of their exposed k -tuples, assuming a k -level generalization.

Straight copy See Fig. 3(b). Tiles are added to copy the pattern along one end of a region through to a parallel end of an adjacent region being constructed. This rule is deterministic.

Turn copy See Fig. 3(c). Tiles are added to copy the pattern along one end of a region to a perpendicular end of an adjacent region being constructed. Counters will be required to position the tiles appropriately to complete this deterministic rule.

The algorithm begins by assembling a random pattern string that will be copied to the top and bottom of the box. Then random patterns are added for the remaining edges of the box, and these are copied to the corresponding edges accordingly. (Refer to Fig. 1 for corresponding edges.) Finally, the shape that will fold is cut out by raising the temperature, assuming that the bonds between tiles along the region-borderline have weak threshold temperatures. The regions that are shaded in Fig. 3(a) are cut away.

3.2 Notation

All of our algorithms will use a 3-level concatenation generalization model; thus a tile is a 4-tuple of triplets that we write $T_N \times T_S \times T_W \times T_E$, with each $T_i = (\sigma_1, \sigma_2, \sigma_3)$. We change the order of the tuple to more easily identify tiles that will bind, since most binding will be north-south or west-east. (This decision is arbitrary and purely notational.) We assume all tiles are oriented so that the directions are clear.

We define the set $\Pi \subset \Sigma$ to be the “random patterns” $\pi_1, \pi_2, \dots, \pi_p$ used as components of exposed triplets for tiles used in random assembly; their use will become clear when we discuss implementation of random assembly below.

We use counters to control the growth of our planar shape. This concept has been well explored in [7] and earlier papers. Each tile can be assigned a

position in the plane denoted by a horizontal and vertical coordinate. We create symbols for position counters and then allow tiles to hybridize if the positions match, creating a mechanism for algorithms to place tiles in absolute or relative positions. Let $H(i)$ and $V(j)$ be the symbols denoting horizontal position i and vertical position j , respectively.

3.3 Row-by-Row Algorithm

Summary After random assembly of the base strip, we use a row-by-row straight copy using a one-dimensional counter through regions **A-D** to copy the base strip's pattern. We then use straight copy to fill in the bodies of regions **E** and **F** and add the edges using random assembly, as these will correspond to other portions of the shape. We then use a turn copy through **G-J** to make those edges correspond. Finally, we do a sequence of straight and turn copies from **E** and **F** through **K-N** and **O-R** to complete the shape. We then raise the temperature to cut away the shaded regions.

Implementation of the self-assembly patterns are discussed briefly below. A complete discussion, including the specific tiles added for each step, can be found in the full version of the paper.

Implementation of Random Assembly The base strip shown in Fig. 3(a) contains the unique pattern that is copied to the edges of the shape. Randomness is achieved by adding tiles of type

$$(\pi_k, \kappa_2, V(0)) \times (\overline{\pi_k}, \overline{\kappa_2}, \overline{V(-1)}) \times (\kappa_1, H(i), V(0)) \times (\overline{\kappa_1}, \overline{H(i+1)}, \overline{V(0)}) \quad (1)$$

where the tiles in (1) vary over all i , $2 \leq i \leq n-2$, and all k , $1 \leq k \leq |II|$. Thus at any position i , a tile with any pattern π_k can adhere, so the final sequence of patterns on the assembled shape will be unique. The use of counters assures the strip will have length n .

Implementation of Straight Copy The straight-copy pattern for a region requires $2n+1$ steps. (Some regions can be done in parallel, so the number of steps is less than $2n+1$ times the number of straight-copy regions.) One counter, in the direction of growth, is used. Tiles are added one row at a time to prevent misformations; at each timestep, only tiles for the current row are added, which ensures (with the help of temperature) that the dominant factor in binding is the pattern sequence (of π_k 's). For example, tiles like the following are added, where the constant $X = (\varphi_1, \varphi_2, \varphi_3)$, for each step i , $1 \leq i \leq 2n-2$ and all patterns $\pi_k \in II$:

$$(\pi_k, \kappa_2, V(i)) \times (\overline{\pi_k}, \overline{\kappa_2}, \overline{V(i-1)}) \times X \times \overline{X} \quad (2)$$

$$(\pi_k, \kappa_2, V(-i)) \times (\overline{\pi_k}, \overline{\kappa_2}, \overline{V(-i-1)}) \times X \times \overline{X} \quad (3)$$

We assume that X binds weaker than κ_2 , so cycling the temperature ensures the tiles are attached on the κ_2 side.

Implementation of Turn Copy The turn-copy step, for example, copying the bottom edge of **E** to the left edge of **D** through **I** so the shape can fold, is done using vertical and horizontal counters, which essentially places a tile in a specific spot. Therefore we can add all the tiles at once to complete the region without possibility of misformation. For the above example region, we would add the following tiles. Let i and j vary such that $-n \leq i \leq -1$ and $-2n + 1 \leq j \leq -n$. For all i, j , add:

$$i = j + (n - 1), (\pi_k, H(i), V(j)) \times (\overline{\kappa_3}, \overline{H(i)}, \overline{V(j - 1)}) \times (\kappa_3, H(i), V(j)) \times (\overline{\pi_k}, \overline{H(i - 1)}, \overline{V(j)}) \quad (4)$$

$$i < j + (n - 1), (\pi_k, H(i), V(j)) \times (\overline{\pi_k}, \overline{H(i)}, \overline{V(j - 1)}) \times (\kappa_3, H(i), V(j)) \times (\overline{\kappa_3}, \overline{H(i - 1)}, \overline{V(j)}) \quad (5)$$

$$i > j + (n - 1), (\kappa_3, H(i), V(j)) \times (\overline{\kappa_3}, \overline{H(i)}, \overline{V(j - 1)}) \times (\pi_k, H(i), V(j)) \times (\overline{\pi_k}, \overline{H(i - 1)}, \overline{V(j)}) \quad (6)$$

As an extra precaution we can set κ_3 binding to be weaker than π_k , and cycle the temperature several times. In addition, we force the encoding of horizontal and vertical counters at the edges where the folding occurs to be the same, so that the sticky ends are in fact complementary.

Analysis of the Row-by-Row Algorithm Proofs for the following can be found in the full version of the paper; most are clear from evaluating, in detail, the steps described above.

Theorem 1. $|\Sigma| = 8n + |II| + O(1)$.

Theorem 2. *The algorithm has time complexity approximately $5n$.*

Theorem 3. *The space complexity of the row-by-row algorithm is approximately $6|II|n^2 + 10|II|n + 4|II| + 8n$ tiles.*

Theorem 4. *The number of distinct temperatures required is 3.*

Theorem 5. *The misformation probability of row-by-row is 0.*

Clearly, this algorithm does not take advantage of all the parallelism possible with self-assembly and has a rather large time complexity. The full version of the paper discusses other algorithms where fewer steps are required (in particular, straight copy is not done row-by-row), but also gives analysis showing that such algorithms have an increased misformation probability.

4 Conclusion

Our paper introduces a precise extension to the Tile Assembly Model [7] that allows greater information content per tile and scalability to three dimensions. The model better formalizes the abstraction of DNA tiles to symbols, introduces five complexity measures to analyze algorithms, and is the first to extend nanostructure fabrication to three dimensions.

In addition, our paper opens up wide-ranging avenues of research.

First of all, it may be possible to encode information on tiles more succinctly than our algorithms do to accomplish the copy patterns discussed. The existence of good 2-level or 1-level generalization algorithms is unknown.

Algorithms to form other 3D structures, having various applications in biology and computation, can be studied. More work also must be done to quantify the probabilities specified in the paper (possibly including a free-energy analysis of tile binding).

Finally, there remain some important biological issues. In particular, design of a strong tile suitable for our method of computation and design of a 3D building block are two important steps to increasing the feasibility of 3D self-assembly. The use of temperature may be further refined and exploited to improve some complexity results and the number of steps needed in the lab.

References

1. J. Chen and N. C. Seeman. The synthesis from DNA of a molecule with the connectivity of a cube. *Nature*, 350:631–633, 1991. 429, 430
2. A. Condon, R. M. Corn, and A. Marathe. On combinatorial DNA word design. In Winfree and Gifford [12]. 433, 436
3. T.-J. Fu and N. C. Seeman. DNA double-crossover molecules. *Biochemistry*, 32:3211–3220, 1993. 429, 432
4. T. H. LaBean, E. Winfree, and J. H. Reif. Experimental progress in computation by self-assembly of DNA tilings. In Winfree and Gifford [12]. 430
5. T. H. LaBean, H. Yan, J. Kopatsch, F. Liu, E. Winfree, H. Reif, and N. Seeman. The construction, analysis, ligation and self-assembly of DNA triple crossover complexes. *J. Am. Chem. Soc.*, 122:1848–1860, 2000. 429, 432
6. I. V. Markov. *Crystal Growth for Beginners: Fundamentals of Nucleation, Crystal Growth, and Epitaxy*. World Scientific, Singapore, 1995. 435
7. P. Rothmund and E. Winfree. The program-size complexity of self-assembled squares. In F. F. Yao, editor, *Proceedings of the 32nd Annual ACM Symposium on Theory of Computing*, Portland, OR, 21–23 May 2000. ACM Special Interest Group on Algorithms and Computation Theory. 430, 431, 433, 434, 435, 437, 439
8. N. C. Seeman. Nucleic-acid junctions and lattices. *Journal of Theoretical Biology*, 2:237–247, 1982. 429, 432
9. H. Wang. Proving theorems by pattern recognition. *Bell System Technical Journal*, 40:1–42, 1961. 432
10. E. Winfree. On the computational power of DNA annealing and ligation. In E. B. Baum and R. J. Lipton, editors, *DNA Based Computers*, DIMACS: Series in Discrete Mathematics and Theoretical Computer Science, pages 199–210. American Mathematical Society, May 1995. 430
11. E. Winfree, T. Eng, and G. Rozenberg. String tile models for DNA computing by self-assembly. In A. Condon and G. Rozenberg, editors, *DNA Based Computers VI*, Leiden, The Netherlands, 13–17 June 2000. Leiden Center for Natural Computing. 430
12. E. Winfree and D. Gifford, editors. *Preliminary Proceedings, Fifth International Meeting on DNA Based Computers*, Cambridge, Massachusetts, 14–15 June 1999. DIMACS. 440

13. E. Winfree, F. Liu, L. A. Wenzler, and N. C. Seeman. Design and self-assembly of two-dimensional DNA crystals. *Nature*, 394:539–544, 1998. 430